

Logika i algebra Boole'a

Matura rozszerzona z informatyki – Sesja 3

Kacper Gębusia

- 1 Zmienne logiczne, operatory, tablice prawdy
- 2 Prawa algebry Boole'a i upraszczanie wyrażeń
- 3 Wyrażenie z tablicy prawdy, bramka NAND
- 4 Półsumator i sumator
- 5 Operatory bitowe i logiczne w Pythonie
- 6 Ćwiczenia

- 1 Dodaj binarnie: $1011 + 1110 = ?$
- 2 Odejmij binarnie: $10110 - 01011 = ?$
- 3 Zapisz -100 w kodzie U2 na 8 bitach.
- 4 Zamień $0,375_{10}$ na system dwójkowy (zapis bitowy).

Zmienna logiczna przyjmuje jedną z dwóch wartości: **1 (prawda)** lub **0 (fałsz)**.

Operator	Zapis	Kiedy daje 1?
NOT	$\neg a$ lub $\sim a$	gdy $a = 0$
AND	$a \wedge b$	gdy oba argumenty są 1
OR	$a \vee b$	gdy co najmniej jeden jest 1
XOR	$a \oplus b$	gdy argumenty są różne
NAND	$\neg(a \wedge b)$	gdy nie oba są 1
NOR	$\neg(a \vee b)$	gdy oba są 0
Implikacja	$a \rightarrow b$	zawsze, oprócz $a = 1, b = 0$
Równoważność	$a \leftrightarrow b$	gdy argumenty są równe

Pomocne: $a \rightarrow b = \neg a \vee b$; $a \oplus b = (a \wedge \neg b) \vee (\neg a \wedge b)$

Tablice prawdy – NOT, AND, OR

NOT

a	$\neg a$
0	1
1	0

AND

a	b	$a \wedge b$
0	0	0
0	1	0
1	0	0
1	1	1

OR

a	b	$a \vee b$
0	0	0
0	1	1
1	0	1
1	1	1

Tablice prawdy – XOR, NAND, NOR

XOR

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

NAND

a	b	NAND
0	0	1
0	1	1
1	0	1
1	1	0

NOR

a	b	NOR
0	0	1
0	1	0
1	0	0
1	1	0

Tablice prawdy – implikacja i równoważność

Implikacja ($a \rightarrow b$)

a	b	$a \rightarrow b$
0	0	1
0	1	1
1	0	0
1	1	1

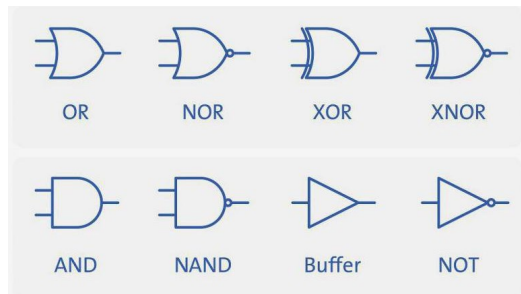
Równoważność / XNOR

a	b	$a \leftrightarrow b$
0	0	1
0	1	0
1	0	0
1	1	1

Priorytet operatorów: najpierw NOT, potem AND, potem OR (jak potęgi/mnożenie/dodawanie). W razie wątpliwości – nawiasy.

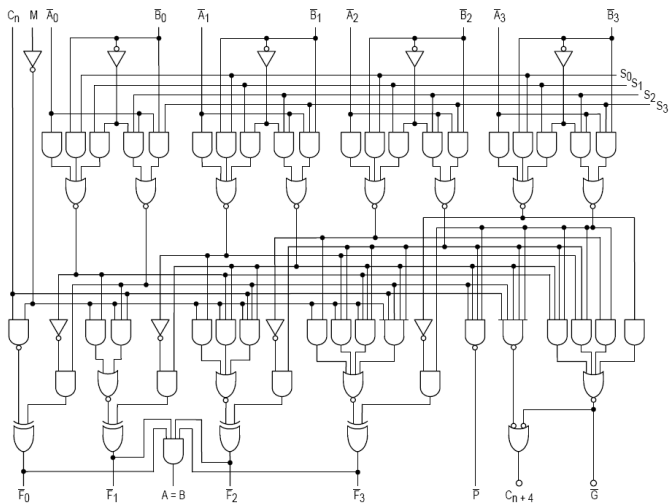
Bramki logiczne na schematach

W rzeczywistości wyrażenia występują jako **bramki logiczne**. Na schematach bramki są oznaczane jako:



Czytamy schemat od wejść ku wyjściu: wyjście każdej bramki zapisujemy jako wyrażenie i podstawiamy do następnej.

Przykładowy schemat układu



Rysunek: Schemat logiczny układu 74181 - 4-bitowej jednostki arytmetycznej (ALU)

Tablica prawdy dla wyrażenia złożonego

Tablica dla n zmiennych ma 2^n wierszy. Wypełniamy kolumnami pośrednimi – po jednej na każdą podformułę.

Przykład 1: $f = (a \vee b) \wedge \neg c$

a	b	c	$a \vee b$	$\neg c$	f
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	0	0
1	1	0	1	1	1
1	1	1	1	0	0

Podstawowe prawa algebry Boole'a

Prawo	Zapis
Przemienność	$a \wedge b = b \wedge a, \quad a \vee b = b \vee a$
Łączność	$(a \wedge b) \wedge c = a \wedge (b \wedge c)$
Rozdzielność	$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ $a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$
Elementy neutralne	$a \wedge 1 = a, \quad a \vee 0 = a$
Elementy dominujące	$a \wedge 0 = 0, \quad a \vee 1 = 1$
Idempotentność	$a \wedge a = a, \quad a \vee a = a$

Uwaga: w algebrze Boole'a **obie** rozdzielności działają (w zwykłej arytmetyce tylko jedna) – częste źródło błędów.

Prawa specjalne: dopełnienie, absorpcja, de Morgan

Prawo	Zapis
Dopełnienie	$a \wedge \neg a = 0, \quad a \vee \neg a = 1$
Podwójna negacja	$\neg \neg a = a$
Absorpcja	$a \vee (a \wedge b) = a, \quad a \wedge (a \vee b) = a$
de Morgana	$\neg(a \wedge b) = \neg a \vee \neg b$ $\neg(a \vee b) = \neg a \wedge \neg b$

Dowód absorpcji: $a \vee (a \wedge b) = (a \wedge 1) \vee (a \wedge b) = a \wedge (1 \vee b) = a \wedge 1 = a$

Sprawdzenie de Morgana dla $(a, b) = (1, 0)$: $\neg(1 \wedge 0) = 1$ oraz $\neg 1 \vee \neg 0 = 0 \vee 1 = 1 \checkmark$

Przykłady upraszczania (1/2)

Przykład 2: $(a \wedge b) \vee (a \wedge \neg b)$

$$= a \wedge (b \vee \neg b) = a \wedge 1 = \mathbf{a}$$

Przykład 3: $a \vee (\neg a \wedge b)$

$$= (a \vee \neg a) \wedge (a \vee b) = 1 \wedge (a \vee b) = \mathbf{a \vee b}$$

Przykład 4: $\neg(\neg a \vee \neg b)$

$$= \neg\neg a \wedge \neg\neg b = \mathbf{a \wedge b}$$

Przykłady upraszczania (2/2)

Przykład 5: $(a \vee b) \wedge (a \vee \neg b)$

$$= a \vee (b \wedge \neg b) = a \vee 0 = \mathbf{a}$$

Przykład 6: $(a \vee b) \wedge \neg a$

$$= (a \wedge \neg a) \vee (b \wedge \neg a) = 0 \vee (\neg a \wedge b) = \neg \mathbf{a} \wedge \mathbf{b}$$

Przykład 7: $\neg(a \vee b) \vee (\neg a \wedge b)$

$$= (\neg a \wedge \neg b) \vee (\neg a \wedge b) = \neg a \wedge (\neg b \vee b) = \neg \mathbf{a}$$

Suma iloczynów (SOP)

Metoda: dla **każdego wiersza z wynikiem 1** budujemy iloczyn wszystkich zmiennych (wprost, jeśli w wierszu ma 1, zanegowaną, jeśli ma 0). Iloczyny łączymy sumą.

Przykład 8 (głosowanie większościowe):

<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Upraszczanie funkcji większościowej

$$f = (\neg a \wedge b \wedge c) \vee (a \wedge \neg b \wedge c) \vee (a \wedge b \wedge \neg c) \vee (a \wedge b \wedge c)$$

Wykorzystujemy $a \wedge b \wedge c$ trzykrotnie (idempotentność):

- $(\neg a \wedge b \wedge c) \vee (a \wedge b \wedge c) = b \wedge c$
- $(a \wedge \neg b \wedge c) \vee (a \wedge b \wedge c) = a \wedge c$
- $(a \wedge b \wedge \neg c) \vee (a \wedge b \wedge c) = a \wedge b$

$$f = (a \wedge b) \vee (a \wedge c) \vee (b \wedge c)$$

czyli $f = 1$, gdy **co najmniej dwie** zmienne są równe 1.

NAND jako bramka uniwersalna

Za pomocą samej bramki NAND ($a \uparrow b = \neg(a \wedge b)$) można zbudować każdą inną:

- $\neg a = a \uparrow a$
- $a \wedge b = (a \uparrow b) \uparrow (a \uparrow b)$
- $a \vee b = (a \uparrow a) \uparrow (b \uparrow b)$ (z prawa de Morgana)

Sprawdzenie AND dla $a = 1, b = 1$: $a \uparrow b = 0$, więc $0 \uparrow 0 = 1$ ✓ (i $1 \wedge 1 = 1$).

Półsumator

W Sesji 2 dodawaliśmy binarnie kolumna po kolumnie. Pojedynczą kolumnę realizują układy z bramek.

Półsumator dodaje dwa bity a, b : daje sumę S i przeniesienie C .

a	b	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = a \oplus b, \quad C = a \wedge b \quad (\text{bo } 1 + 1 = 10_2)$$

Pełny sumator

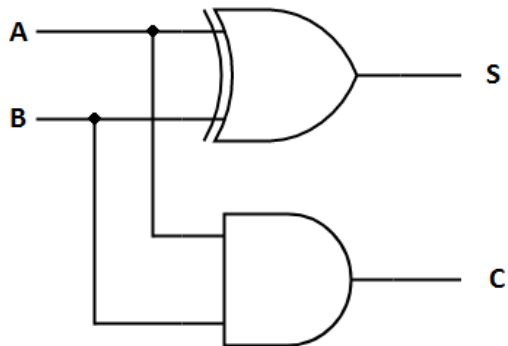
Dodaje trzy bity: a , b oraz przeniesienie z poprzedniej kolumny c_{in} .

a	b	c_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

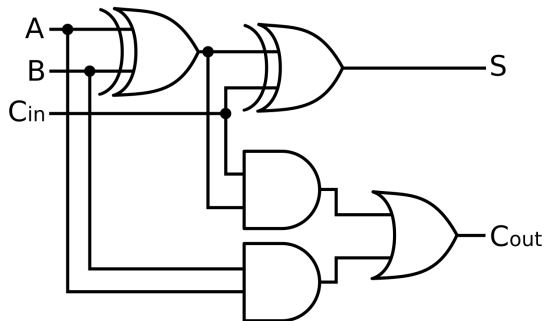
$$S = a \oplus b \oplus c_{in}, \quad C_{out} = (a \wedge b) \vee (a \wedge c_{in}) \vee (b \wedge c_{in})$$

To dokładnie funkcja większościowa z Przykładu 8! Łącząc pełne sumatory w szereg, dostajemy sumator wielobitowy.

Schematy sumatorów



Rysunek: Półsumator



Rysunek: Pełny sumator

Operatory bitowe i logiczne w Pythonie

Operator	Znaczenie	Przykład ($12=1100_2$, $10=1010_2$)
$a \& b$	AND bit po bicie	$12 \& 10 = 8$
$a b$	OR bit po bicie	$12 10 = 14$
$a \wedge b$	XOR bit po bicie	$12 \wedge 10 = 6$
$\sim a$	negacja bitów	$\sim 12 = -13$
$a \ll k$	przesunięcie w lewo ($\cdot 2^k$)	$12 \ll 2 = 48$
$a \gg k$	przesunięcie w prawo ($\div 2^k$)	$12 \gg 2 = 3$

Dlaczego ~ 12 daje -13 ? W U2 negacja wszystkich bitów to $-x - 1$.

Pułapka priorytetów: $\&$, $|$, $\wedge$ mają niższy priorytet niż $==$

Zawsze piszemy $(x \& 1) == 0$, nie $x \& 1 == 0$.

Typowe zastosowania

```
x = 45                # 101101 w systemie dwojkowym
print(x & 1)           # 1  -> liczba nieparzysta (ostatni bit)
print(x & 15)          # 13 -> ostatnie 4 bity (to samo co x % 16)
print((x >> 2) & 1)    # 1  -> bit nr 2 (liczac od 0 od prawej)
print(x | (1 << 3))    # 45 -> ustawienie bitu nr 3 (tu juz byl 1)
print(x ^ (1 << 3))    # 37 -> odwrocenie bitu nr 3
```

Sprawdzenie: $45 = 101101_2$. Odwrócenie bitu nr 3 daje $100101_2 = 37$ ✓

Generowanie i sprawdzanie tablic prawdy w Pythonie

```
from itertools import product

for a, b in product([0, 1], repeat=2):
    print(a, b, "|", int(not (a and b)))    # tablica NAND
```

Sprawdzanie równoważności (prawo de Morgana):

```
for a, b in product([0, 1], repeat=2):
    lewa  = not (a and b)
    prawa = (not a) or (not b)
    print(a, b, lewa == prawa)    # zawsze True
```

Ćwiczenia na zajęciach (1/3)

- 1 Zbuduj tablicę prawdy dla $(a \vee \neg b) \wedge c$.
- 2 Czy wyrażenia $\neg(a \vee b)$ oraz $\neg a \wedge \neg b$ są równoważne? Uzasadnij tablicą prawdy.
- 3 Uprość:
a) $a \wedge (\neg a \vee b)$ b) $\neg a \vee (a \wedge b)$

Ćwiczenia na zajęciach (2/3)

- 4 Zapisz wyrażenie dla funkcji trzech zmiennych, równej 1 **tylko** gdy $a = b = c$.
- 5 Zbuduj $a \wedge b$ używając wyłącznie bramek NAND i sprawdź dla $a = 1, b = 0$.
- 6 Pełny sumator: podaj S i C_{out} dla $a = 1, b = 0, c_{in} = 1$.

Ćwiczenia na zajęciach (3/3)

- 7 Podaj wyniki: `25 & 12`, `25 | 12`, `25 ^ 12`, `~25`.
- 8 Podaj wyniki: `45 & 15` oraz `(45 » 3) & 1`.
- 9 Co wypisze program? `print(6 & 3, 6 | 3, 6 ^ 3)`

- 1 Zbuduj tablicę prawdy dla $\neg a \vee (b \wedge c)$.
- 2 Wykaż, że $a \rightarrow b$ jest równoważne $\neg b \rightarrow \neg a$.
- 3 Uprość:
a) $(a \vee b) \wedge (\neg a \vee b)$ b) $\neg(a \wedge \neg b) \vee b$ c) $a \wedge (a \vee b) \wedge (a \vee c)$

- 4 Zapisz wyrażenie na podstawie tablicy prawdy (zmienne a, b, c , kolejność wierszy 000...111):
a) $f = 1$ tylko w wierszach 001, 011, 101, 111 b) $f = 1$ tylko w wierszach 000 i 010.
Uprość wyniki.
- 5 Zbuduj $a \vee b$ używając wyłącznie bramek NAND i sprawdź dla $(a, b) = (0, 0)$ oraz $(1, 0)$.
- 6 Zsumuj $1011_2 + 0110_2$ kolumna po kolumnie, podając dla każdej kolumny S i C_{out} .

- 7 **(Python)** a) Co wypisze `print(13 & 6, 13 | 6, 13 ^ 6)`? b) Zapisz warunek parzystości `x` operatorem bitowym. c) Ustaw bit nr 3 w `x` na 1; sprawdź dla `x = 5`. d) Zapisz `x * 8` przez przesunięcie bitowe.
- 8 Ile wierszy ma tablica prawdy wyrażenia o 5 zmiennych? Ile o 10?

Dziękuję za uwagę!